IFN636: Software Life Cycle Management:

Assessment 1.1

Total Marks 15

Assessment name: Software requirements analysis and design (Full-Stack CRUD Application Development with DevOps Practices)

We have provided this template for you. Include all parts of this assessment in one file (this file), convert it to PDF and then submit it via Canvas before the deadline.

The first page (this page: cover page) of your assessment file should include following information:

Mark Distribution:

	Marks
• Design Diagrams Using SysML	4
• Project Management with JIRA	4
• UI/UX Design Using Figma	4
• Report	3
	Total Marks: 15

Your final mark will depend on your demonstration, which will be used to verify the functionality, completeness, and your understanding of the implemented system. During the demonstration, you will be required to clearly show how your project works and explain the key features and design decisions.

In addition, the use of Gen-AI tools must be clearly reported in your report. If you have used any Gen-AI tools during the development of your project (e.g., drawing diagram, design and design ideas, documentation, or debugging), you must include a section explaining how and where Gen-AI was used. Failure to report the use of Gen-AI may be considered a breach of academic integrity requirements.

Full name: SHINO VARGHESE

Student ID: N12118702

Tutor's name (tutor's full name): Natasha Tanzila Monalisa

Tutorial day and time: Thursday 9:00 to 11:00

This document is copyright protected. Do not copy. Do not share (online and offline).

Draw.io Links:

1. Block Definition Diagram
2. Requirement Diagram
3. Activity Diagram
4. Usecase Diagram

JIRA Link

Figma Link

(Please make sure your links are working to avoid negative marking)

This document is copyright protected. Do not copy. Do not share (online and offline).

1. Project Overview

Devolenz Software License Management System

The **Devolenz Software License Management System** is designed for organisational use to efficiently manage, monitor, and control software licenses. The system provides a centralised platform for maintaining software license records and ensures that software resources are distributed among employees in a structured and controlled manner.

The system involves two primary participants:

- **Administrator (IT Department)** – responsible for managing software licenses, employee accounts, and license requests
- **User (Employees)** – organisational staff who access software through licenses assigned by the administrator

The overall workflow ensures that software licenses are allocated appropriately while maintaining an accurate and up-to-date license inventory.

Project Scope

The project scope has been defined to focus on core functionalities such as license management, user management, request handling, and license assignment. These features ensure that the system meets its primary objective of managing software licenses efficiently.

1. Software License Inventory Setup

Before software licenses can be assigned, the administrator must first register them in the system.

The administrator logs into the system and accesses the **License Management module**, where software details such as software name, total number of licenses, purchase date, and expiry date are entered. The system stores this information in the database and automatically calculates the number of available licenses.

This step ensures that all software licenses are centrally managed and tracked within the organisation.

2. New Employee Account Creation

When a new employee joins the organisation, the administrator creates a user account through the **User Management module**.

The administrator enters employee details such as name, department, and job role. The system stores this information and activates the account for system access.

This document is copyright protected. Do not copy. Do not share (online and offline).

This process ensures that only authorised users can access the system.

3. Role-Based License Assignment

After account creation, the administrator may assign software licenses based on the employee's role.

For example, developers may receive development tools, designers may receive design software, and data analysts may receive analytics tools.

The administrator selects the employee and assigns the appropriate software license. The system verifies license availability before assignment and updates the available license count accordingly.

This ensures that employees are equipped with the necessary tools required for their roles.

4. Employee Login and Access

Once the account is created, employees can log into the system using their credentials.

Upon successful authentication, the employee is directed to the **User Dashboard**, where all assigned licenses are displayed. The dashboard includes details such as software name, license type, expiry date, and license status.

This enables employees to easily access and monitor their assigned software resources.

5. Employee Requests Additional Software License

If employees require additional software, they can submit a request through the **Request Software License page**.

The employee selects the required software and provides justification for the request. The system records the request in the database and sets its status to **Pending**.

This feature allows employees to request tools necessary for their tasks in a structured manner.

6. Administrator Reviews License Requests

The administrator reviews all submitted requests through the **License Request Management module**.

The system displays pending requests along with relevant details such as employee name, requested software, and license availability. Based on this information, the administrator decides whether to approve or reject the request.

This document is copyright protected. Do not copy. Do not share (online and offline).

This step ensures that license allocation aligns with organisational requirements and policies.

7. License Approval and Assignment

If a request is approved, the administrator proceeds to assign the requested license.

The system verifies license availability before assignment. If available, the license is allocated to the employee, the available license count is updated, and the request status is changed to **Approved**.

The employee can then view the assigned license in their dashboard.

8. Employee Views Assigned Licenses

Employees can view all assigned licenses through the **User Dashboard**.

The system retrieves and displays license details including software name, expiry date, and status. This provides clear visibility of authorised software usage.

9. Administrator Monitors and Manages Licenses

The administrator continuously monitors license usage across the organisation.

This includes viewing all licenses, tracking assignments, monitoring availability, and identifying licenses nearing expiration. This ensures efficient utilisation and control of software resources.

10. Administrator Deletes Expired Licenses

To maintain accurate records, the administrator removes expired licenses from the system.

The administrator identifies expired licenses in the **License Management module** and deletes them after confirmation. The system updates the database and license inventory automatically.

This ensures that the system maintains accurate and up-to-date license records.

Out of Scope

The following features are not included in the current implementation to reduce system complexity and ensure timely completion of the project. These can be considered as future enhancements.

1. Notification System

- Real-time notifications for request updates or license assignment are not implemented

This document is copyright protected. Do not copy. Do not share (online and offline).

- Users and administrators track updates through the system interface instead

2. Advanced Authentication and Security

- Features such as multi-factor authentication, single sign-on (SSO), and biometric authentication are not included

3. Integration with External Systems

- No integration with third-party tools (e.g., HR systems, cloud platforms, or external APIs)

4. Automated License Renewal

- Automatic renewal of licenses before expiry is not implemented
- License renewal is handled manually by the administrator

5. Advanced Reporting and Analytics

- Detailed analytics, forecasting, and reporting dashboards are not included

6. Multi-Organisation Support

- The system is designed for a single organisation only
- Multi-tenant or multi-organisation support is not implemented

This document is copyright protected. Do not copy. Do not share (online and offline).

2. Real-World Application

In modern organisations, particularly in IT and technology-driven environments, employees rely on a wide range of software tools such as development platforms, design applications, and data analytics tools to perform their daily tasks. Managing these software licenses efficiently is a critical responsibility of the IT department.

In real-world scenarios, organisations often face challenges such as over-allocation of licenses, under-utilisation of software, expired licenses, and lack of visibility into license usage. Manual management of software licenses can lead to increased operational costs and potential compliance risks.

To address these challenges, organisations use enterprise-level solutions such as **ServiceNow IT Asset Management**, which provide centralised control over software assets and license allocation. These systems allow IT administrators to maintain an inventory of software licenses, track usage, and manage distribution across employees.

For example, when a new employee joins an organisation, the IT department creates a user account and assigns software licenses based on the employee's role. Developers may be provided with development tools, designers with graphic design software, and analysts with data analytics platforms. If additional software is required, employees submit a request through the system, which is reviewed and approved by the administrator before the license is assigned.

Similarly, platforms like **Microsoft Intune** are used to manage software access and enforce organisational policies across devices. These tools ensure that software is securely distributed and only accessible to authorised users.

Such systems provide significant benefits, including improved visibility of software usage, better control over license allocation, reduced costs, and enhanced compliance with licensing policies. They also streamline communication between employees and administrators, ensuring that software requirements are fulfilled efficiently.

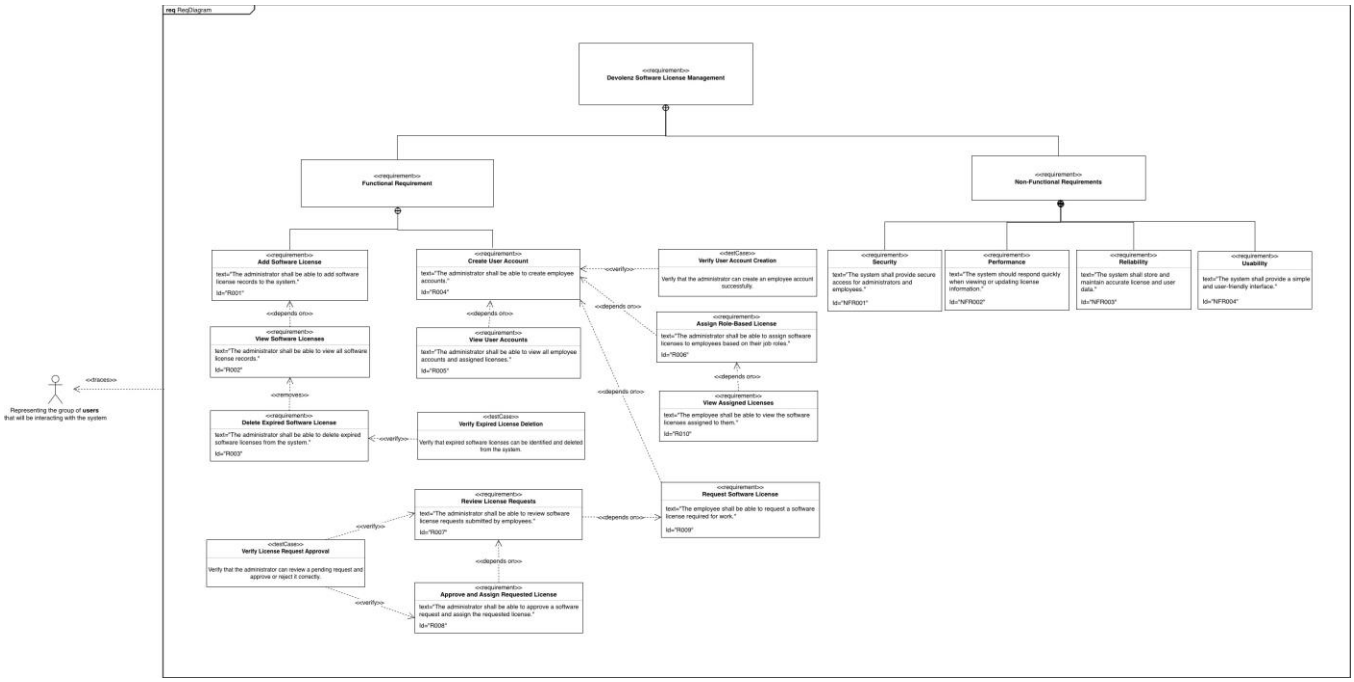
This document is copyright protected. Do not copy. Do not share (online and offline).

3. Project Management and Design

3.1 SysML Design Diagrams

3.1.1 Requirement Diagram

The Requirement Diagram presents a structured representation of the system requirements and their relationships. It illustrates how the overall system requirement is decomposed into **functional and non-functional requirements**, and how these requirements are interconnected using SysML relationships such as **«refine», «depends on», «verify», and «traces»**.



The diagram provides a clear understanding of system behaviour, dependencies, and validation mechanisms.

System Requirement

The top-level requirement is defined as:

- **Devolenz Software License Management**
  The system shall manage software licenses, user accounts, license requests, and license assignments within an organisation.

This requirement is decomposed into two main categories:

- Functional Requirements
- Non-Functional Requirements

Queensland University of Technology
This document is copyright protected. Do not copy. Do not share (online and offline).

Functional Requirements

The functional requirements define the core operations of the system. These include:

- **R001 – Add Software License**
- **R002 – View Software Licenses**
- **R003 – Delete Expired Software License**
- **R004 – Create User Account**
- **R005 – View User Accounts**
- **R006 – Assign Role-Based License**
- **R007 – Review License Requests**
- **R008 – Approve and Assign Requested License**
- **R009 – Request Software License**
- **R010 – View Assigned Licenses**

These requirements collectively support the full workflow of license management, user management, and request handling within the system.

Non-Functional Requirements

The non-functional requirements define system quality attributes:

- **NFR001 – Security**
  Ensures secure access for administrators and users
- **NFR002 – Performance**
  Ensures fast system response during operations
- **NFR003 – Reliability**
  Ensures accurate and consistent data handling
- **NFR004 – Usability**
  Ensures a simple and user-friendly interface

These requirements support the proper functioning and usability of the system.

Relationships Between Requirements

The **«refine» relationship** is used to decompose the main system requirement into detailed functional and non-functional requirements.

The top-level requirement is refined into:

- Functional Requirements
- Non-Functional Requirements

This document is copyright protected. Do not copy. Do not share (online and offline).

This provides a hierarchical structure for the system requirements.

The **«depends on» relationship** represents logical dependencies between requirements.

- **R002 (View Software Licenses)** depends on **R001 (Add Software License)**
- **R003 (Delete Expired Software License)** depends on **R002 (View Software Licenses)**
- **R006 (Assign Role-Based License)** depends on **R004 (Create User Account)**
- **R008 (Approve and Assign Requested License)** depends on **R007 (Review License Requests)**
- **R007 (Review License Requests)** depends on **R009 (Request Software License)**
- **R010 (View Assigned Licenses)** depends on **R006 (Assign Role-Based License)**

These dependencies ensure that system operations follow a logical sequence.

The **«verify» relationship** connects requirements to test cases, ensuring that each requirement can be validated.

Test cases included in the diagram:

- **Verify User Account Creation** → verifies **R004 (Create User Account)**
- **Verify Expired License Deletion** → verifies **R003 (Delete Expired Software License)**
- **Verify License Request Approval** → verifies **R008 (Approve and Assign Requested License)**

These test cases ensure that critical system functionalities are correctly implemented and testable.

The **«traces» relationship** links system requirements to actors.

- The actor represents users interacting with the system
- Requirements are traced to user interactions such as:
    - Creating accounts
    - Requesting licenses
    - Viewing assigned licenses

This ensures traceability between system functionality and user involvement.

The Requirement Diagram is designed to provide a clear and structured representation of system requirements. The use of hierarchical decomposition through «refine»

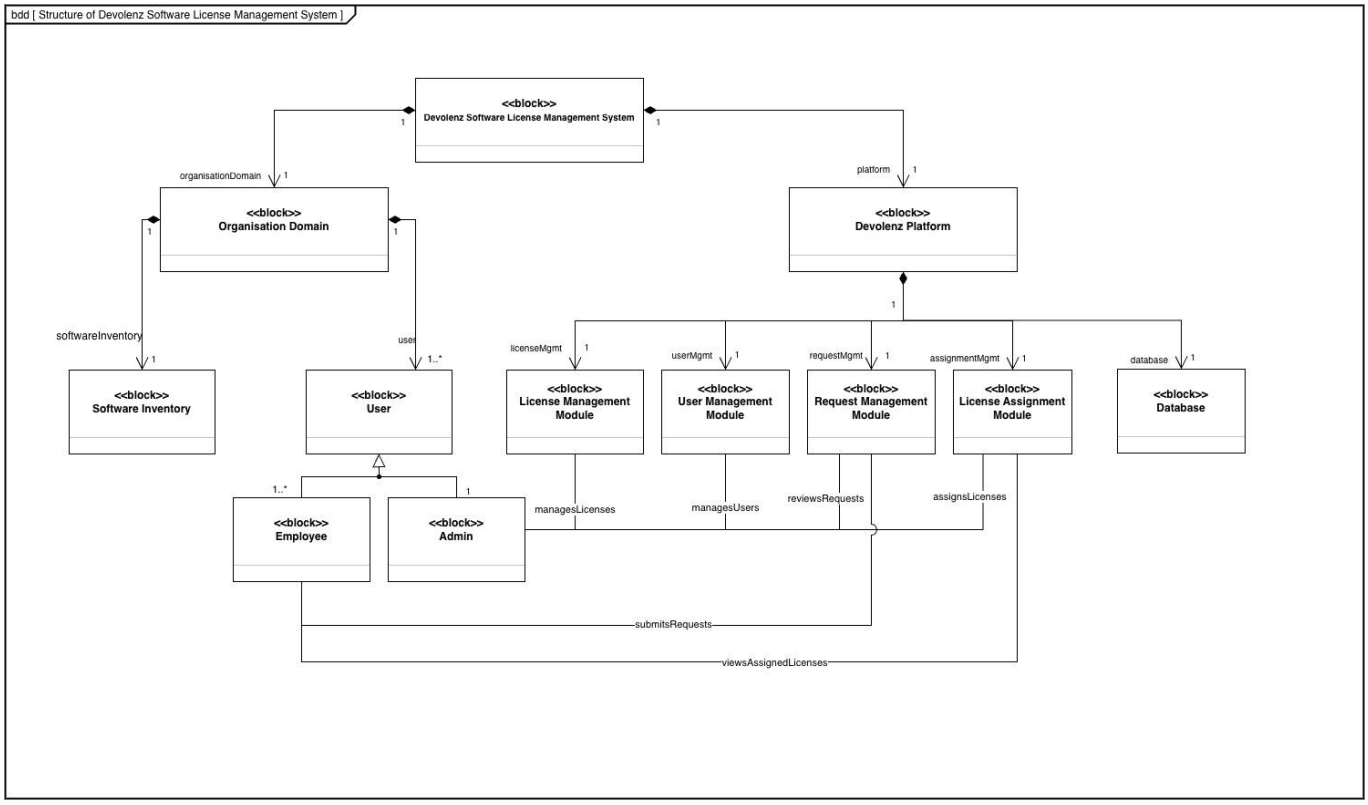
Queensland University of Technology

This document is copyright protected. Do not copy. Do not share (online and offline).

improves readability, while «depends on» relationships clearly represent logical dependencies between functionalities. The inclusion of «verify» relationships ensures that requirements are testable, and «traces» relationships improve traceability between users and system behaviour.

3.1.2 Block Definition Diagram

The Block Definition Diagram (BDD) represents the **structural architecture** of the system by defining the main system components (blocks), their relationships, and multiplicities. It provides a high-level view of how different parts of the system are organised and interact with each other.



The diagram illustrates the decomposition of the system into domain components, platform modules, and user roles.

The top-level block is:

- **Devolenz Software License Management System**

This represents the overall system and is composed of two primary blocks:

- **Organisation Domain**
- **Devolenz Platform**

This document is copyright protected. Do not copy. Do not share (online and offline).

This separation reflects the real-world structure where the system operates within an organisational environment and is supported by a software platform.

Organisation Domain

The **Organisation Domain** represents the real-world entities interacting with the system. It includes:

- **Software Inventory**
  Stores all software license information within the organisation
- **User**
  Represents individuals interacting with the system

The **User block is further specialised into:**

- **Admin (IT Department)**
- **Employee**

This is modelled using **generalisation (inheritance)**, where both Admin and Employee inherit properties from the User block.

Multiplicity:

- One organisation can have **multiple users (1..*)**
- The system maintains a single software inventory

Devolenz Platform

The **Devolenz Platform** represents the system's internal modules that handle all operations.

It is composed of the following modules:

- **License Management Module**
- **User Management Module**
- **Request Management Module**
- **License Assignment Module**
- **Database**

(Notification Module is excluded as per project scope)

Each module is responsible for a specific part of the system functionality.

This document is copyright protected. Do not copy. Do not share (online and offline).

License Management Module

- Manages software licenses
- Supports adding, viewing, and deleting licenses

User Management Module

- Handles creation and management of user accounts

Request Management Module

- Processes software license requests submitted by employees

License Assignment Module

- Assigns licenses to users based on role or approval

Database

- Stores all system data including users, licenses, and requests

Relationships Between Blocks

The diagram uses different types of relationships to represent system structure:

Composition Relationship

The system is composed of:

- Organisation Domain
- Devolenz Platform

Similarly, the platform is composed of multiple modules.

Composition indicates strong ownership, meaning these components are essential parts of the system.

Association Relationships

Associations represent interactions between users and system modules:

- **Admin → License Management Module** (manages licenses)
- **Admin → User Management Module** (manages users)
- **Admin → Request Management Module** (reviews requests)
- **Admin → License Assignment Module** (assigns licenses)

This document is copyright protected. Do not copy. Do not share (online and offline).

- **Employee → Request Management Module** (submits requests)
- **Employee → License Assignment Module** (views assigned licenses)

Generalisation (Inheritance)

- **User → Admin**
- **User → Employee**

This indicates that Admin and Employee share common user properties but have different responsibilities.

Multiplicity:

Multiplicity defines how many instances of one block relate to another:

- One **Organisation Domain** contains multiple **Users (1..*)**
- One **Platform** includes multiple modules (1)
- Each module interacts with one central **Database (1)**

This ensures clarity in system structure and relationships.

The BDD is designed to clearly separate:

- Real-world entities (Organisation Domain)
- System components (Platform modules)

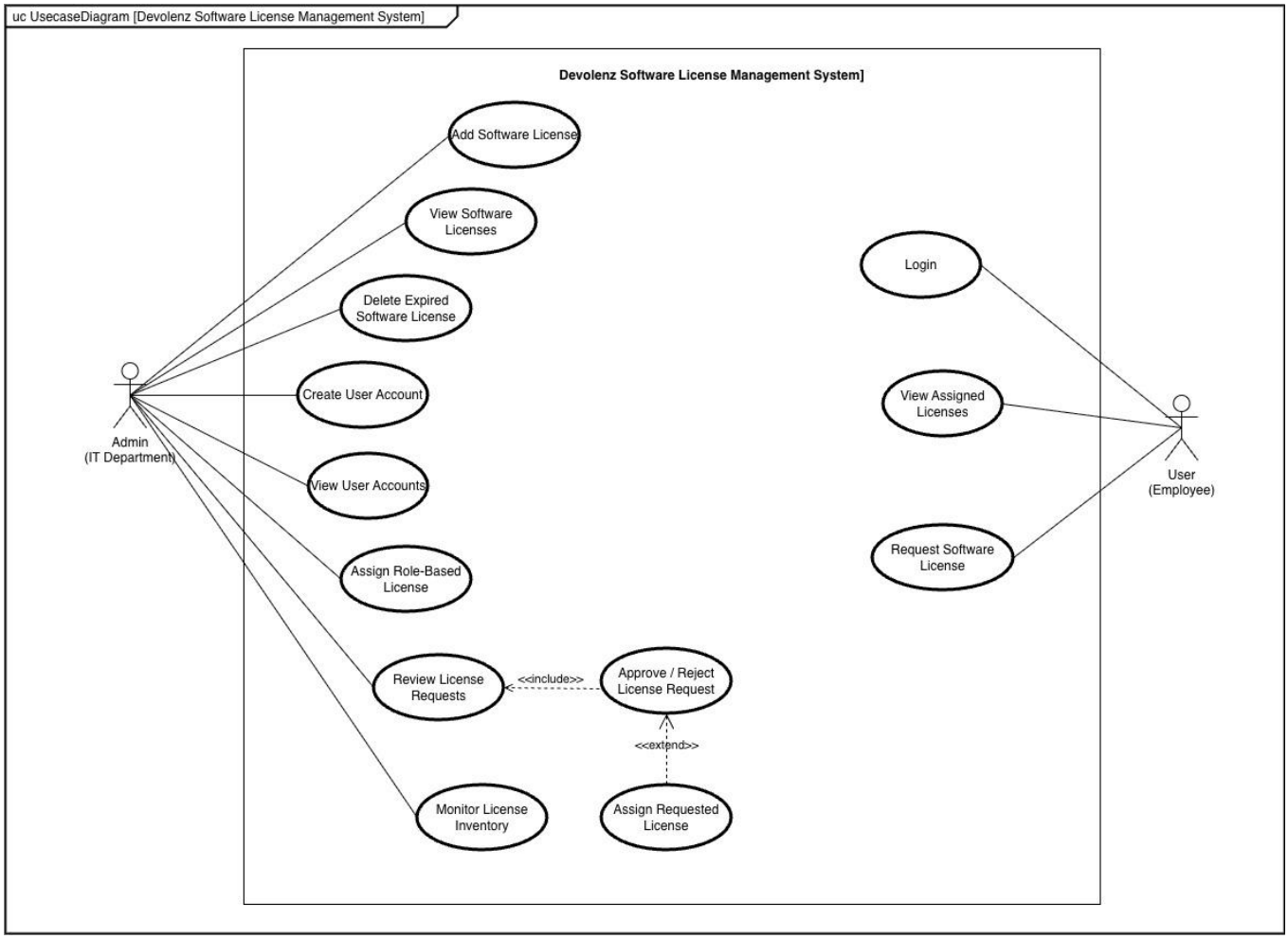
This separation improves clarity and reflects how real systems are structured. The use of modular components ensures scalability, maintainability, and clear responsibility distribution across the system.

Generalisation simplifies user representation, while composition ensures strong structural relationships between system components.

3.1.3 Use Case Diagram

The Use Case Diagram represents the functional behaviour of the system by illustrating how different actors interact with the system to perform specific tasks. It identifies the key operations supported by the system and defines the responsibilities of each actor.

This document is copyright protected. Do not copy. Do not share (online and offline).



The system involves two primary actors:

- **Administrator (IT Department)**
- **User (Employee)**

The **Administrator** is responsible for managing the overall system operations. The administrator can add, view, and delete software licenses to maintain the license inventory. In addition, the administrator manages user accounts by creating and viewing employee records. The administrator also assigns software licenses to users, either based on their roles or by approving individual requests. Furthermore, the administrator reviews software license requests submitted by employees and decides whether to approve or reject them. The administrator also monitors license usage and availability within the organisation.

The **User (Employee)** interacts with the system to access and request software licenses. The user can log into the system, view the licenses assigned to them, and submit requests for additional software required for their job tasks.

This document is copyright protected. Do not copy. Do not share (online and offline).

The relationships between use cases represent dependencies and conditional behaviours. The **"Approve/Reject License Request"** use case includes the **"Review License Requests"** use case, as reviewing is a mandatory step before making a decision. The **"Assign Requested License"** use case extends the approval process, as license assignment occurs only when a request is approved.

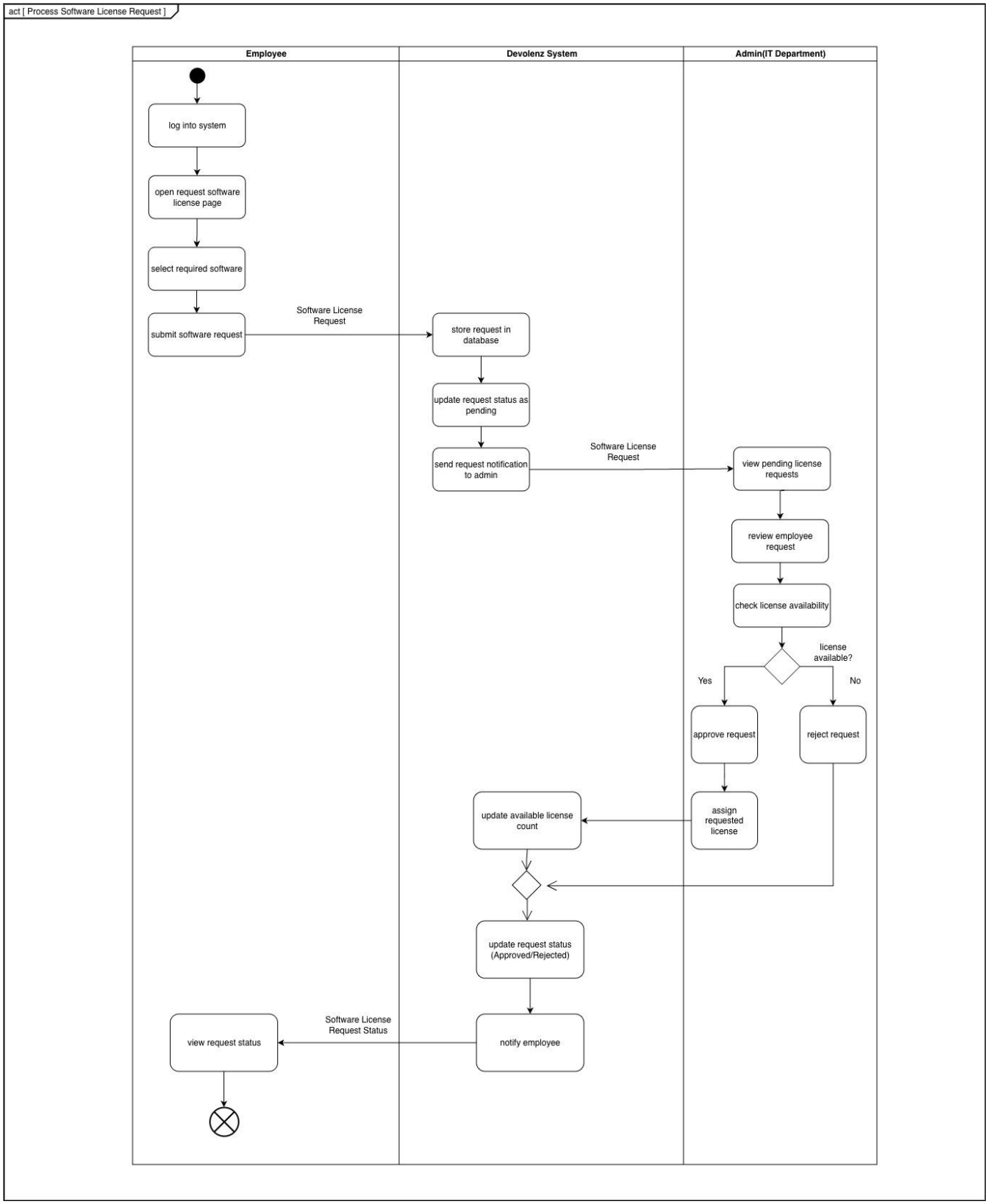
Additionally, the "Assign Role-Based License" use case is not modelled using an «include» relationship with "Create User Account" because account creation is not executed during license assignment. Instead, it is a prerequisite condition, making it a logical dependency rather than a mandatory included behaviour.

The Use Case Diagram provides a clear understanding of system functionality, actor responsibilities, and interaction flow, ensuring that all major operations of the system are properly represented.

3.1.4 Activity Diagram (Software License Request Process)

The Activity Diagram illustrates the workflow of the software license request process within the system. It shows how different participants: **Employee**, **System**, and **Administrator (IT Department)** interact to complete the request lifecycle.

This document is copyright protected. Do not copy. Do not share (online and offline).



The process begins when the **employee logs into the system**, navigates to the software request page, selects the required software, and submits the request. The system then **stores the request in the database**, updates its status to *Pending*, and forwards it to the administrator for review.

This document is copyright protected. Do not copy. Do not share (online and offline).

The **administrator reviews the request** and checks license availability. A decision is then made based on availability:

- If licenses are available, the administrator **approves the request** and assigns the requested license.
- If licenses are not available, the administrator **rejects the request**.

Following the decision, the system **updates the license count (if assigned)** and updates the request status accordingly (Approved or Rejected). Finally, the system **notifies the employee**, and the employee can view the updated request status.

This diagram clearly represents the end-to-end workflow of the license request process, including decision points, system interactions, and responsibilities of each actor. The notification steps in the Activity Diagram are conceptual and used to illustrate system flow. Real-time notification functionality is out of scope, and users can check updates through the system interface.

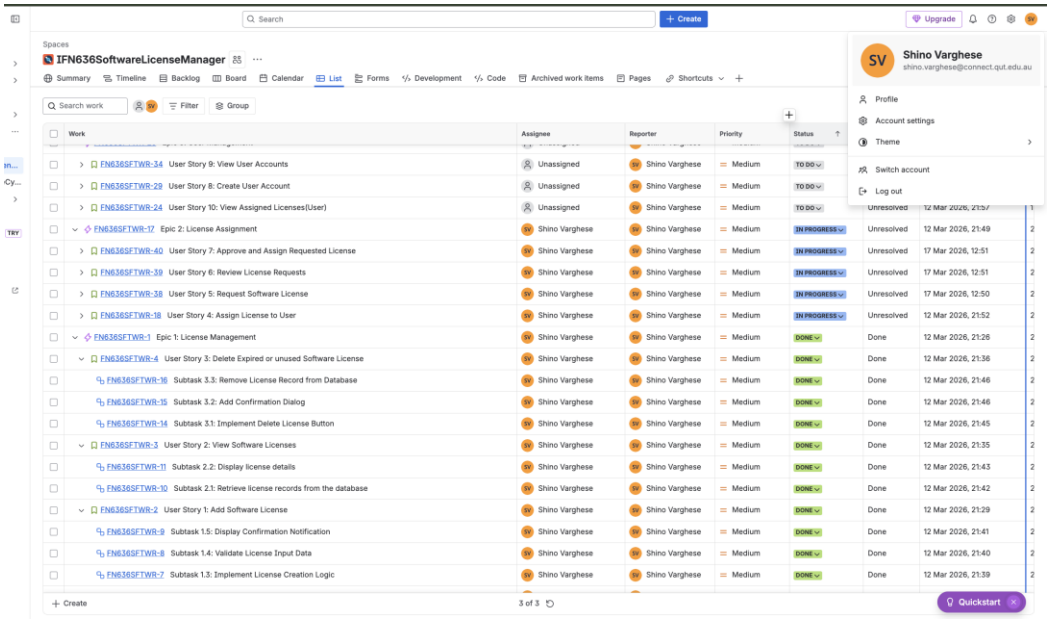
3.2 Software Project Management with JIRA

The project was managed using JIRA to organise, track, and monitor development tasks based on the defined system requirements. The system requirements were translated into structured JIRA issues using **Epics, User Stories, and Subtasks**, ensuring a clear and manageable workflow.

JIRA Board Setup

A Scrum-based JIRA board was created for the assignment. The board consists of workflow columns such as *To Do, In Progress,* and *Done*, which helped in tracking the status of each task throughout the development process.

This document is copyright protected. Do not copy. Do not share (online and offline).



Backlog Creation

The backlog was created by converting system requirements into JIRA issues. The work was organised into three main epics:

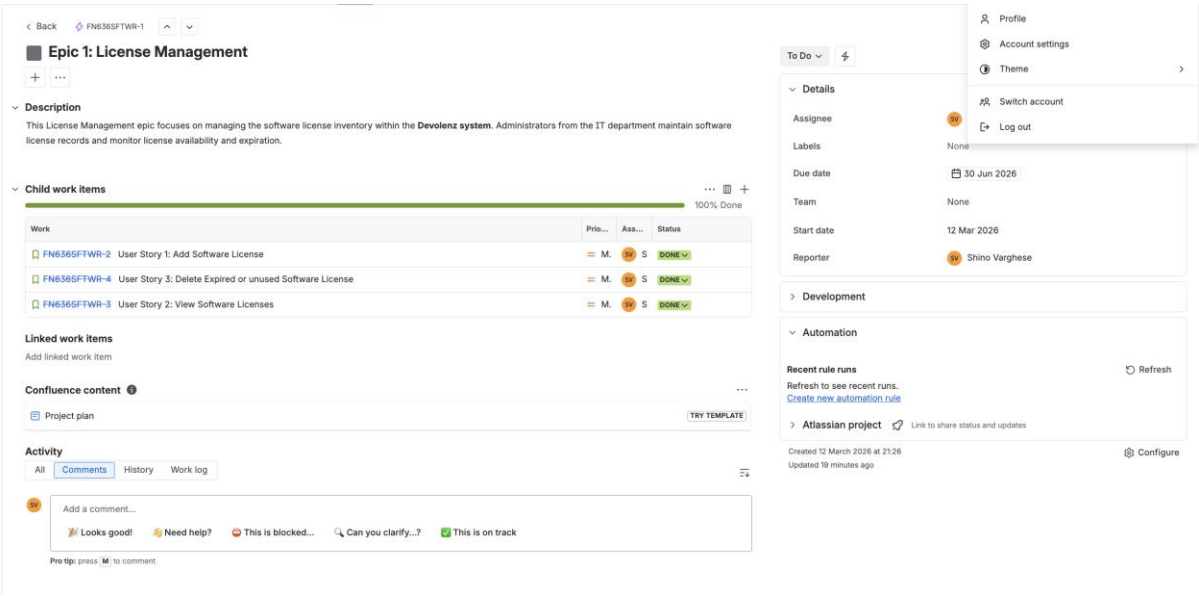
- **Epic 1: License Management** – managing software license records
- **Epic 2: User License Assignment** – handling license allocation and requests
- **Epic 3: User Management** – managing employee accounts

Each epic contains multiple user stories that represent key system functionalities.

Epic 1 – License Management

This epic focuses on managing the software license inventory within the system. The administrator maintains license records, monitors availability, and ensures expired licenses are removed to keep the inventory accurate.

This document is copyright protected. Do not copy. Do not share (online and offline).



User Story 1 – Add Software License

Description

As an administrator, I want to add new software licenses so that the organisation can maintain a centralised record and track license availability.

Subtasks

- Design license input form
- Validate license details (name, type, expiry date)
- Store license data in the database
- Display confirmation after successful creation

User Story 2 – View Software Licenses

Description

As an administrator, I want to view all software licenses so that I can monitor availability and usage.

Subtasks

- Retrieve license records from the database
- Display licenses in a table format
- Show details such as software name, total, assigned, available licenses, and expiry date

This document is copyright protected. Do not copy. Do not share (online and offline).

User Story 3 – Delete Expired Software License

Description

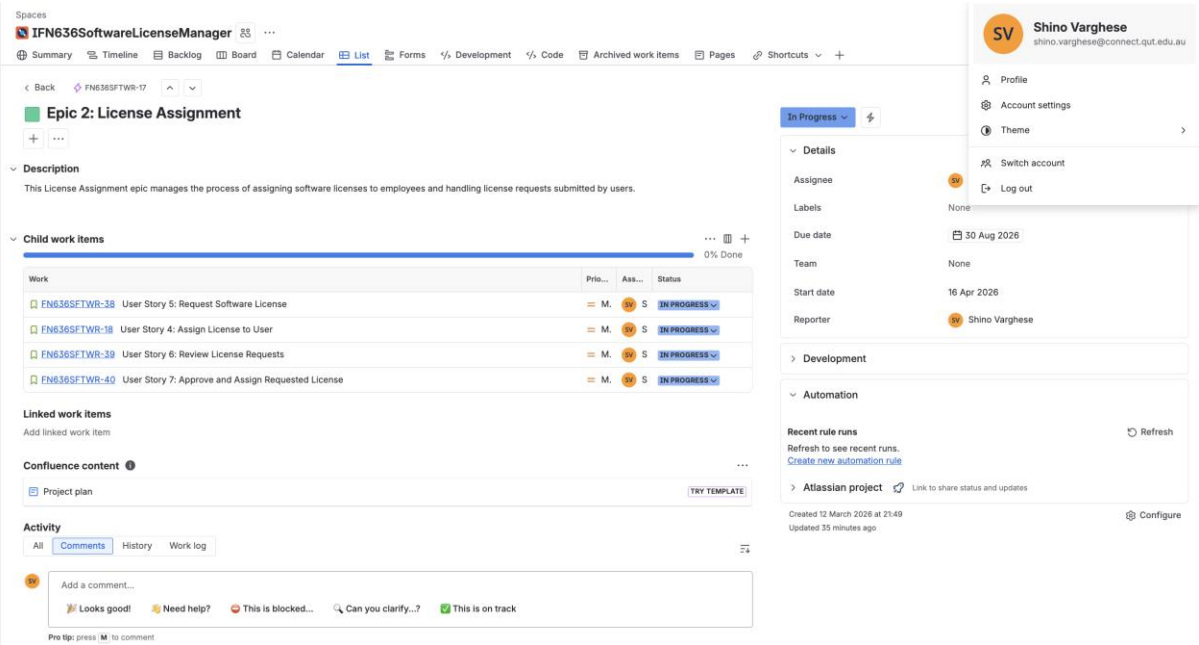
As an administrator, I want to delete expired licenses so that the system maintains accurate and up-to-date records.

Subtasks

- Identify expired licenses based on expiry date
- Provide delete option for expired licenses
- Confirm deletion before removal
- Remove license from database
- Refresh license list

Epic 2 – License Assignment

This epic manages the allocation of licenses to employees and the handling of license requests.



User Story 4 – Assign License to User

Description

As an administrator, I want to assign licenses to employees so that they can access required software tools.

Subtasks

- Create license assignment interface

This document is copyright protected. Do not copy. Do not share (online and offline).

- Retrieve employee list
- Retrieve available licenses
- Assign selected license to employee
- Update license availability

User Story 5 – Request Software License

Description
As an employee, I want to request software licenses so that I can access tools required for my tasks.

Subtasks

- Create request interface
- Retrieve available software list
- Submit license request
- Store request in database
- Update request status to Pending

User Story 6 – Review License Requests

Description
As an administrator, I want to review license requests so that I can approve or reject them based on availability and requirements.

Subtasks

- Retrieve pending requests
- Display request details (employee, software, date)
- Provide approve/reject options
- Update request status

User Story 7 – Approve and Assign Requested License

Description
As an administrator, I want to approve requests and assign licenses so that employees can access requested software.

Subtasks

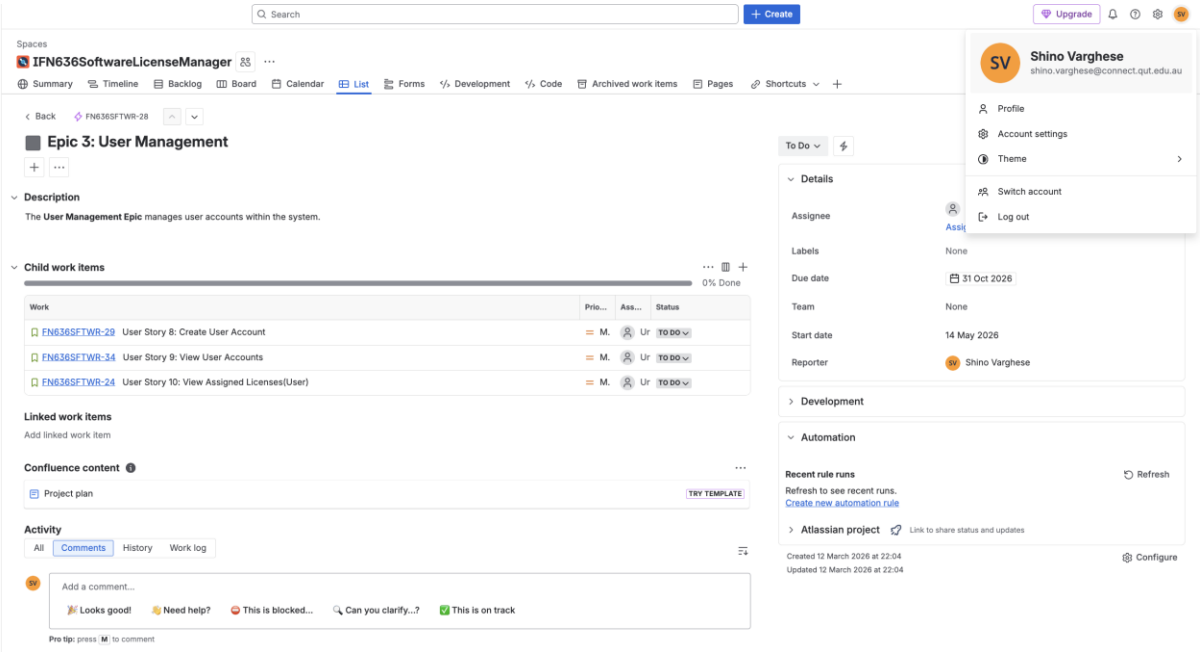
- Check license availability
- Assign license to employee
- Update license count

This document is copyright protected. Do not copy. Do not share (online and offline).

- Update request status to Approved
- Display confirmation

Epic 3 – User Management

This epic focuses on managing employee accounts within the system.



User Story 8 – Create User Account

Description

As an administrator, I want to create user accounts so that employees can access the system.

Subtasks

- Design registration form
- Validate input fields
- Store user data in database
- Display confirmation message

User Story 9 – View User Accounts

Description

As an administrator, I want to view all users so that I can manage accounts and monitor license assignments.

Subtasks

This document is copyright protected. Do not copy. Do not share (online and offline).

- Retrieve user records
- Display user details in table format
- Show assigned licenses for each user

User Story 10 – View Assigned Licenses (User)

Description

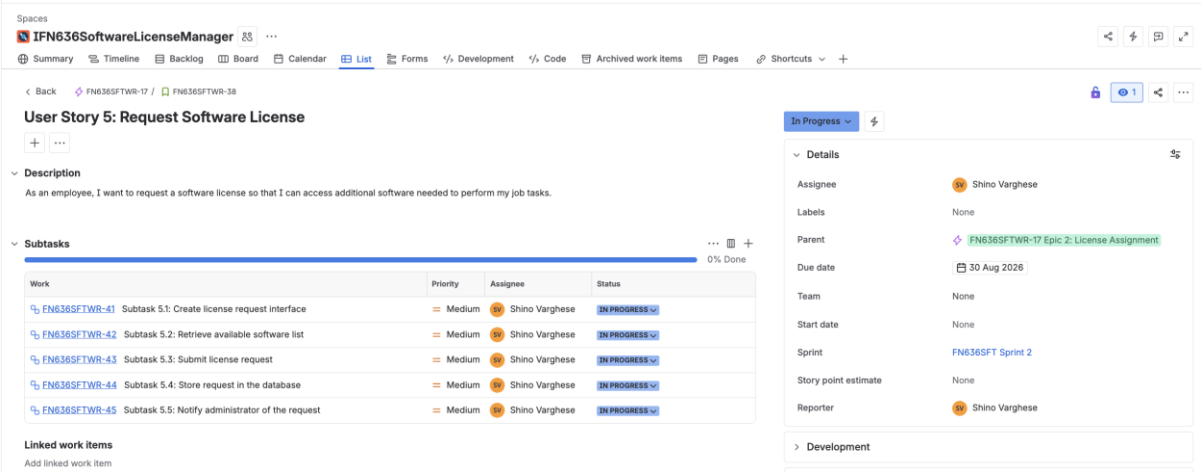
As an employee, I want to view my assigned licenses so that I know which software I am authorised to use.

Subtasks

- Retrieve assigned licenses from database
- Display licenses in user dashboard
- Show license expiry details

Task Assignment

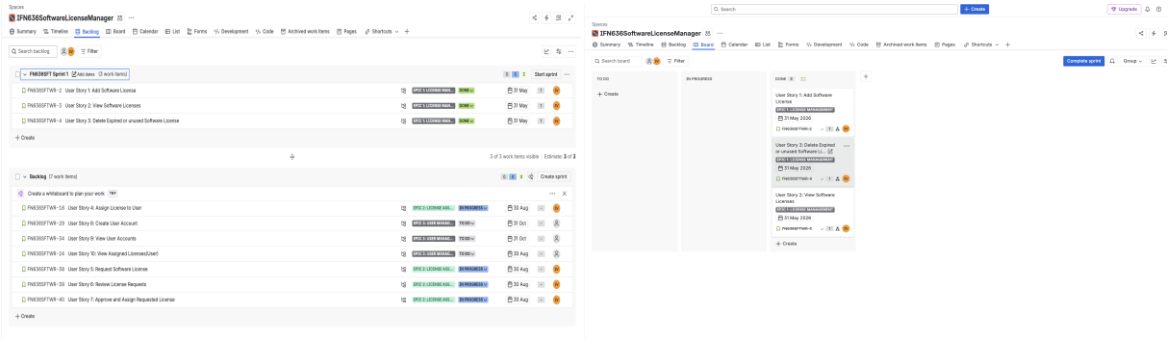
All tasks were assigned to the project member (individual work), ensuring accountability and clear ownership. Each issue includes details such as priority, due date, and status, enabling effective task management.



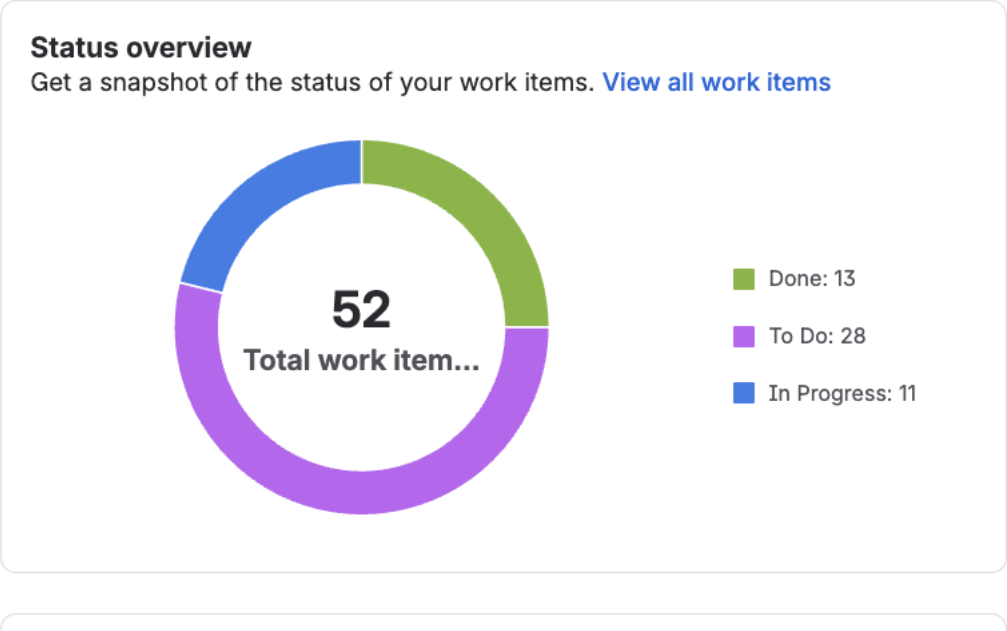
Sprint and Progress Tracking

Tasks were managed through the JIRA board workflow, moving from *To Do* to *In Progress* and finally to *Done*. This allowed continuous monitoring of project progress and ensured timely completion of features.

This document is copyright protected. Do not copy. Do not share (online and offline).



A final overview of the board shows all completed tasks across epics and user stories.

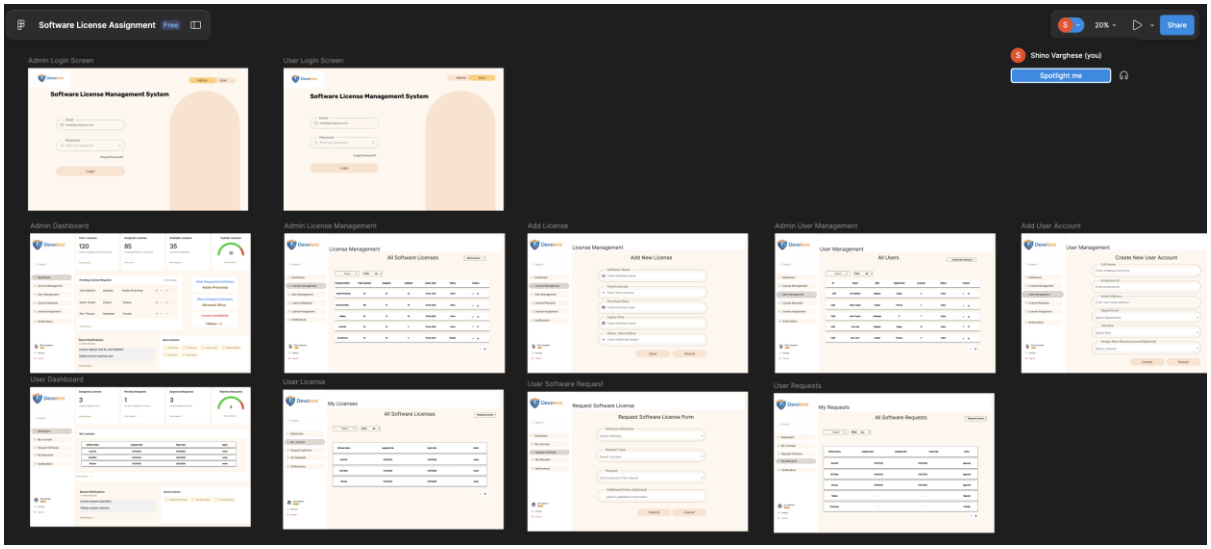


JIRA provided an effective platform to manage the project by organising requirements into structured tasks, tracking progress, and maintaining clear visibility of development activities. It ensured that all system functionalities were properly planned and implemented.

3.3 UI/UX design with Figma

The UI/UX design of the **Software License Management System** focuses on simplicity, clarity, and efficient user interaction. The interface is designed for two main user roles: **Admin** and **User**, ensuring that each role has access to relevant features through a clean and intuitive layout.

This document is copyright protected. Do not copy. Do not share (online and offline).

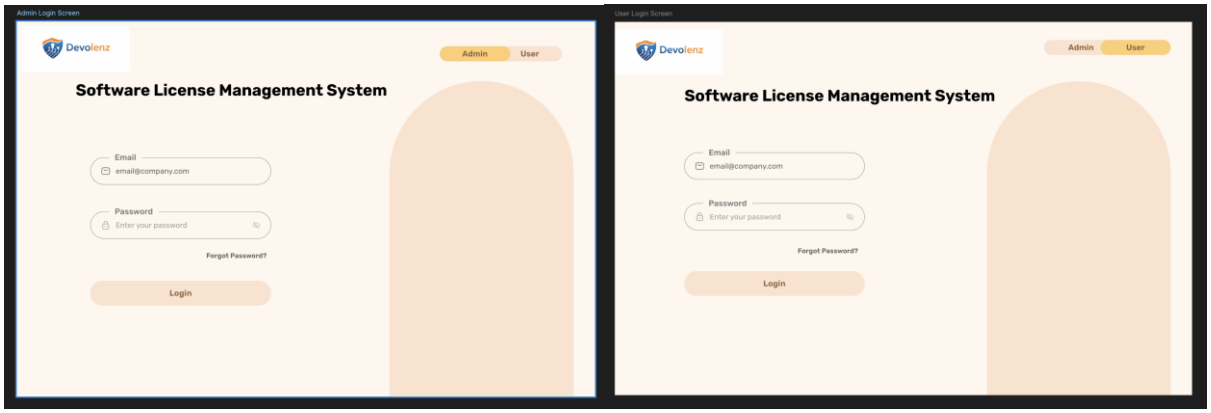


1. Login Screens (Admin & User)

The system provides separate login options for Admin and User roles. The design includes:

- Simple email and password fields
- Clear role selection (Admin/User toggle)
- Minimal distractions with a clean layout

A minimal login interface improves usability and reduces cognitive load, ensuring quick access to the system.



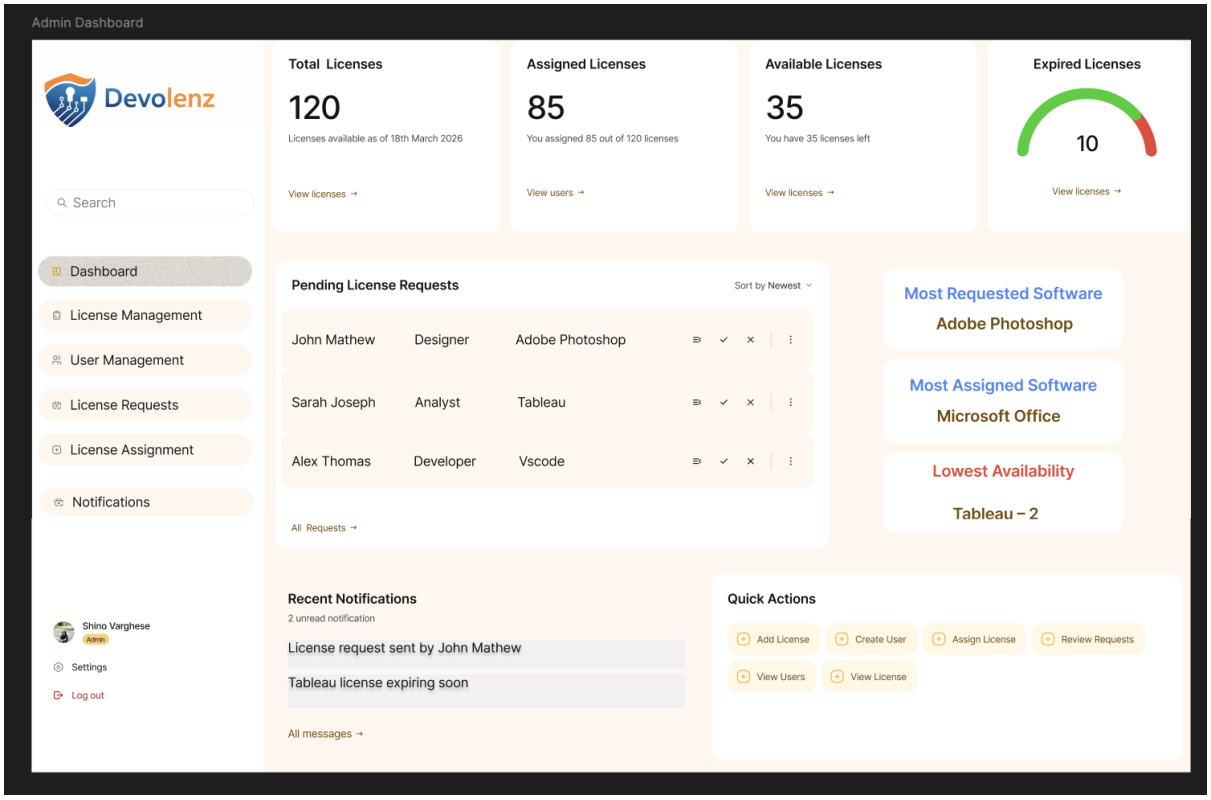
2. Admin Dashboard

The Admin Dashboard provides an overview of system activities:

- Total, assigned, available, and expired licenses
- Pending license requests
- Quick action buttons (Add License, Create User, Assign License)
- Notifications and analytics

This document is copyright protected. Do not copy. Do not share (online and offline).

Card-based layout helps in quick data visualization and decision-making.



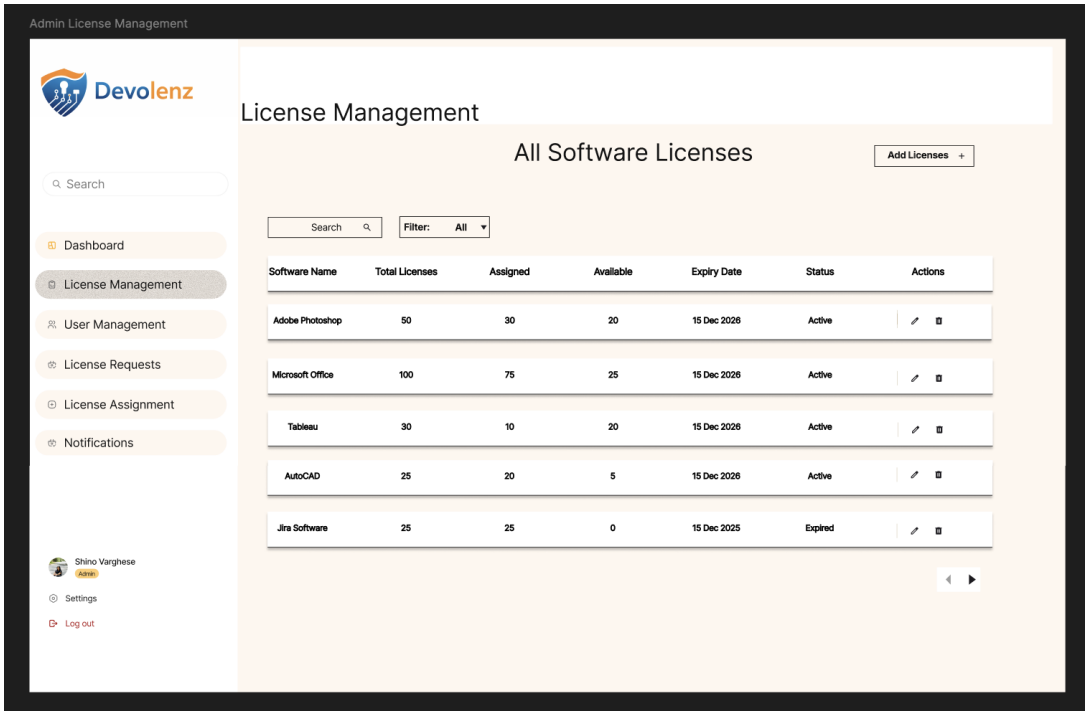
3. License Management (CRUD Operations)

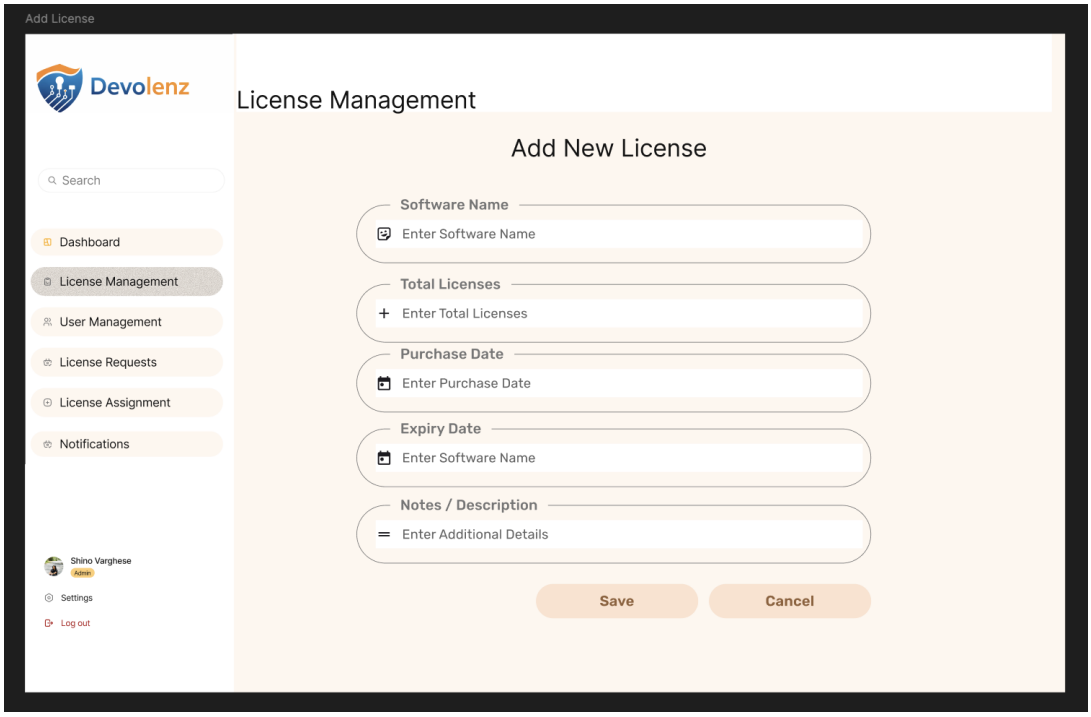
Admins can manage software licenses through:

- Table view with search and filter options
- Add new license form (Create)
- Edit and delete actions (Update/Delete)

Using tables ensures structured data representation, while forms provide easy input for CRUD operations.

This document is copyright protected. Do not copy. Do not share (online and offline).





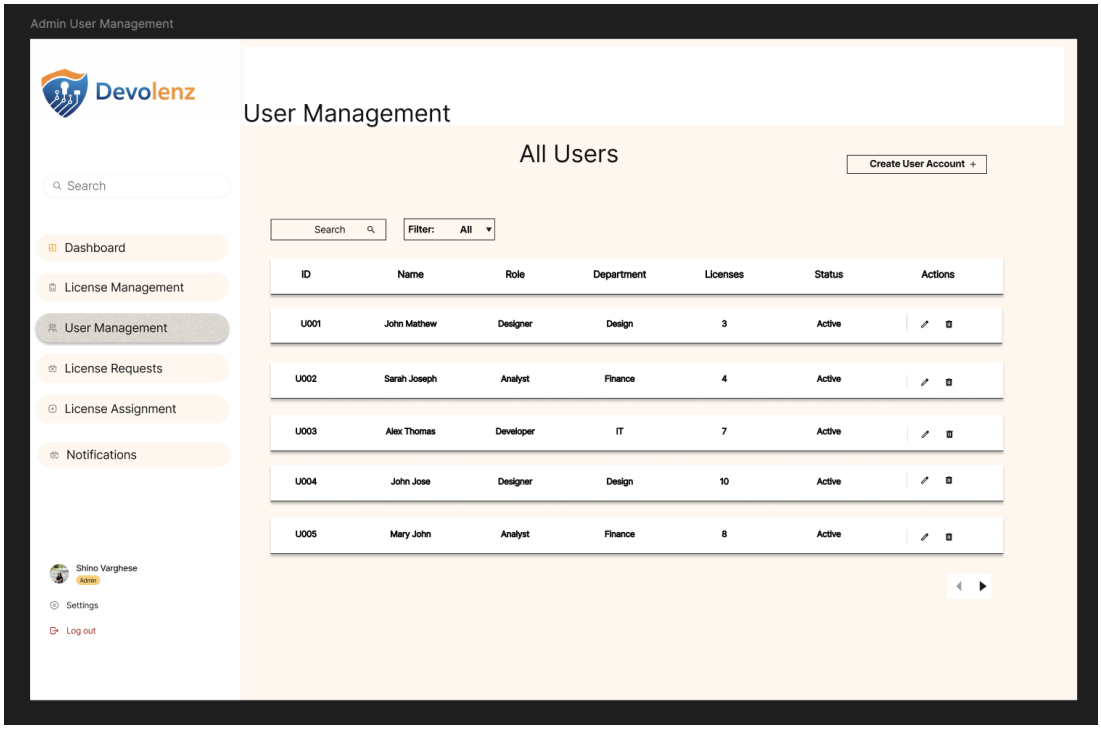
4. User Management

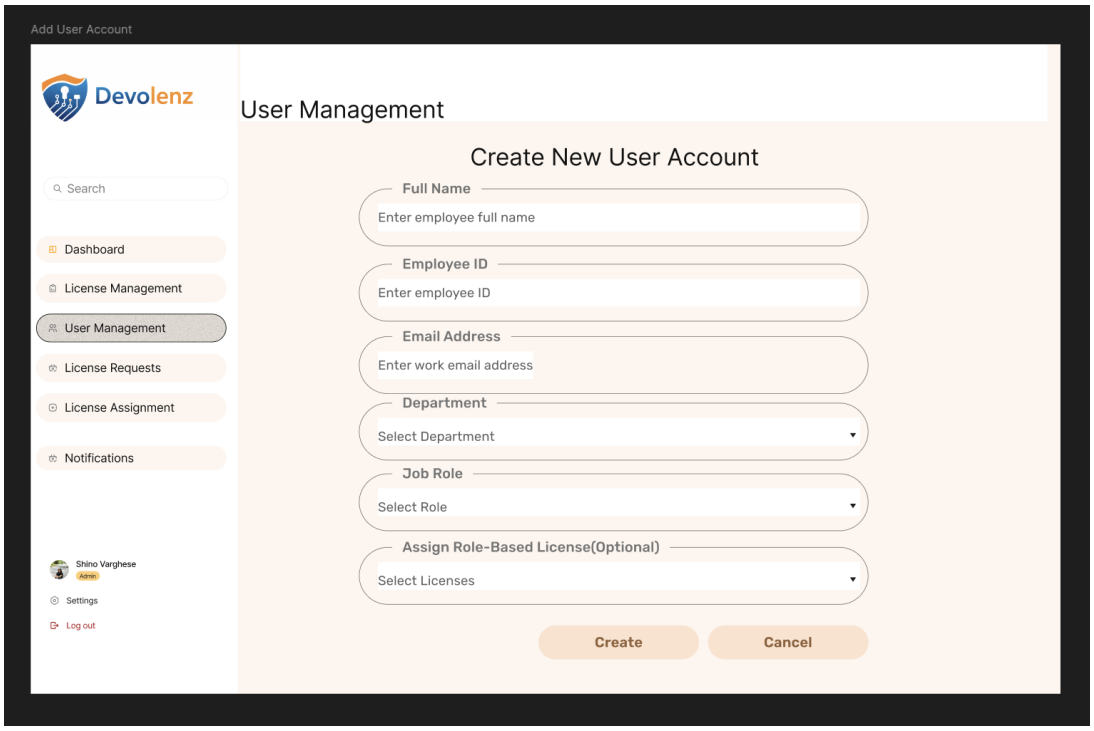
Admins can:

- View all users in a structured table
- Create new user accounts
- Assign roles and departments
- Optionally assign licenses

This document is copyright protected. Do not copy. Do not share (online and offline).

Grouping user details and actions in one interface improves administrative efficiency.





5. License Requests & Assignment

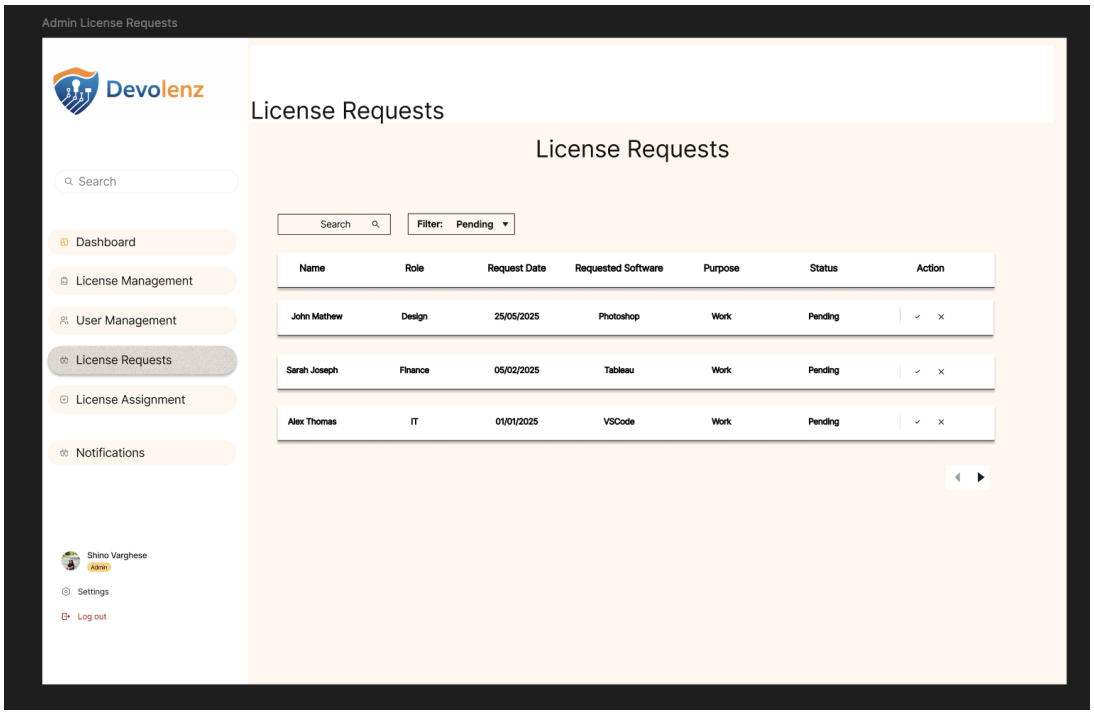
The system supports a request-based workflow:

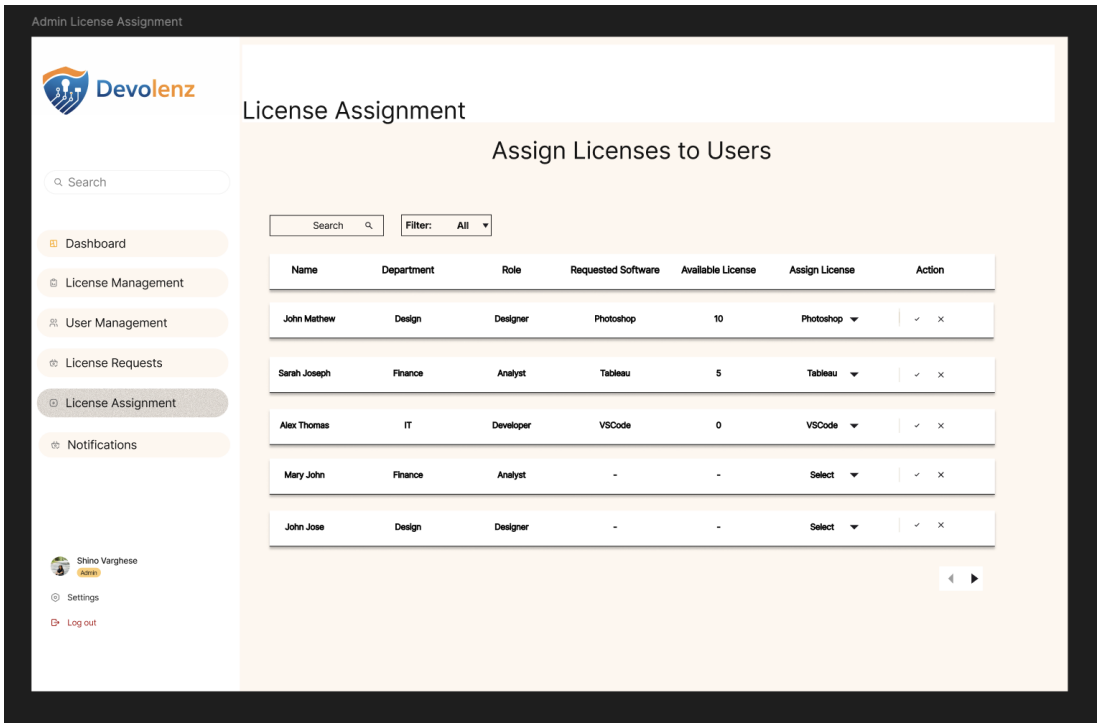
- Users submit license requests
- Admin reviews and approves/rejects requests

This document is copyright protected. Do not copy. Do not share (online and offline).

- License assignment interface with availability tracking

This workflow ensures controlled access and prevents over-allocation of licenses.





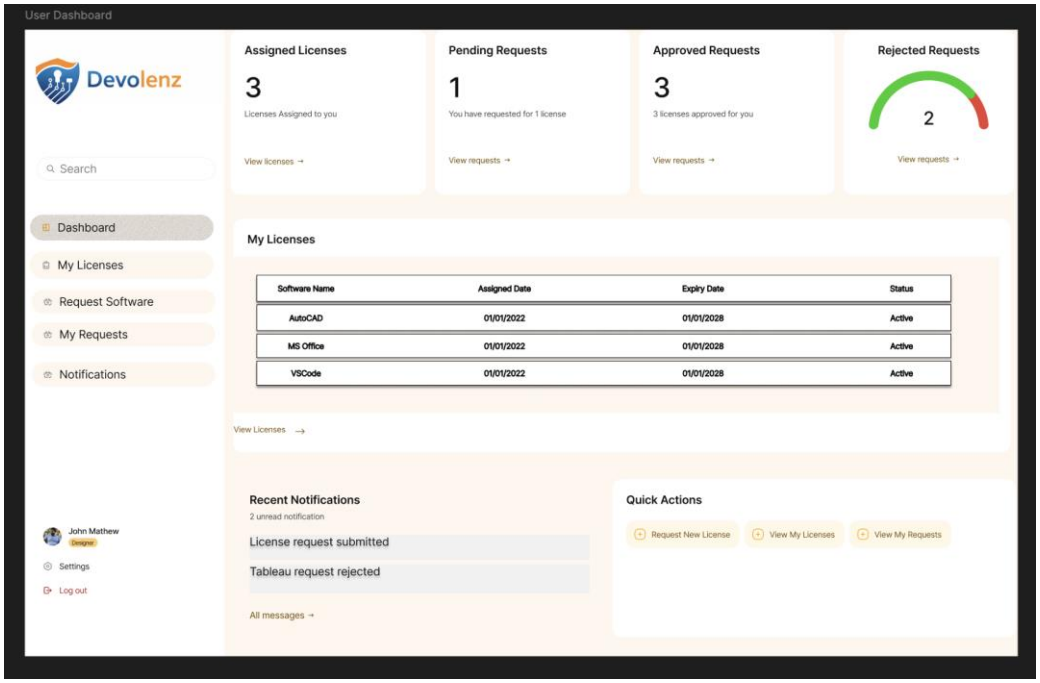
6. User Dashboard

The User Dashboard acts as the central interface for users to monitor their activities. It displays key information such as:

This document is copyright protected. Do not copy. Do not share (online and offline).

- Total assigned licenses
- Pending, approved, and rejected requests
- Quick access links to view licenses and requests
- Recent notifications

The dashboard provides a quick summary of user activity, enabling users to understand their status without navigating multiple pages.



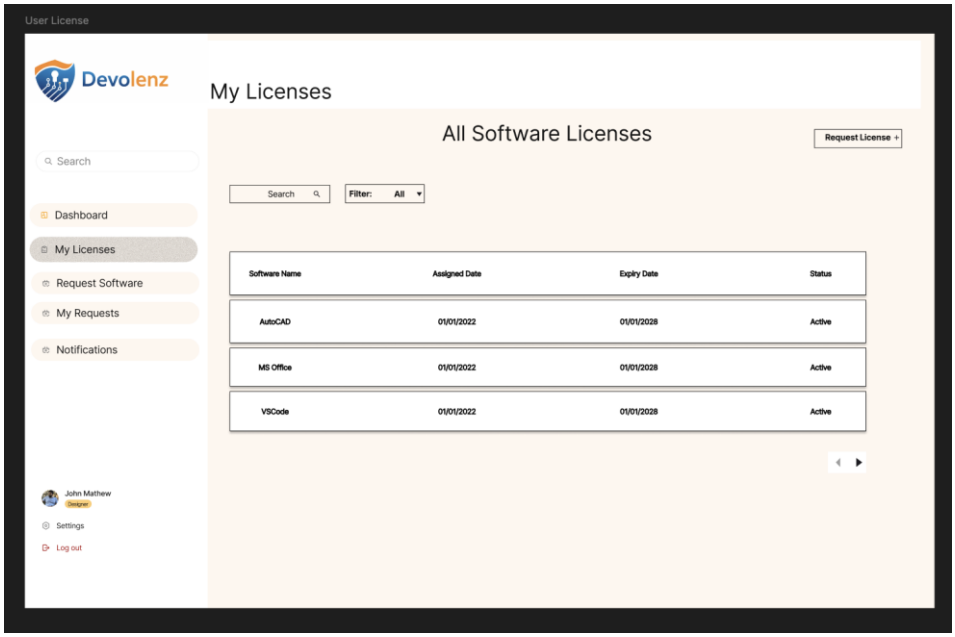
7. My Licenses Page

This page allows users to view all licenses assigned to them. It includes:

- Software name
- Assigned date and expiry date
- License status (Active/Expired)
- Search and filter options

A tabular layout ensures clear visibility of license details and helps users easily track validity and usage.

This document is copyright protected. Do not copy. Do not share (online and offline).

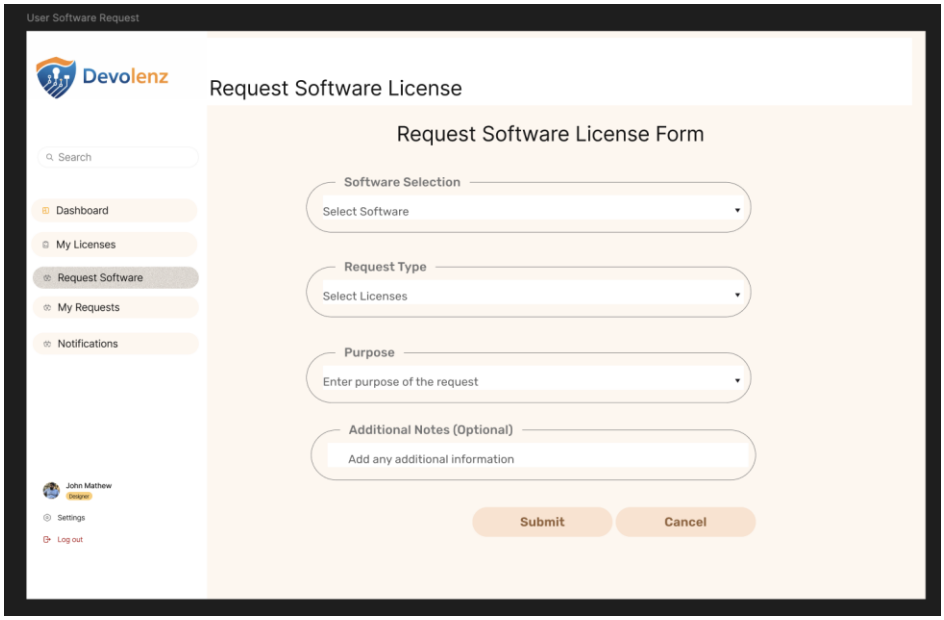


8. Request Software Page

Users can request new software licenses through a structured form:

- Software selection dropdown
- Request type (new/renewal)
- Purpose of request
- Optional notes
- Submit and cancel options

The form is designed to be simple and guided, ensuring users provide all required information for approval.



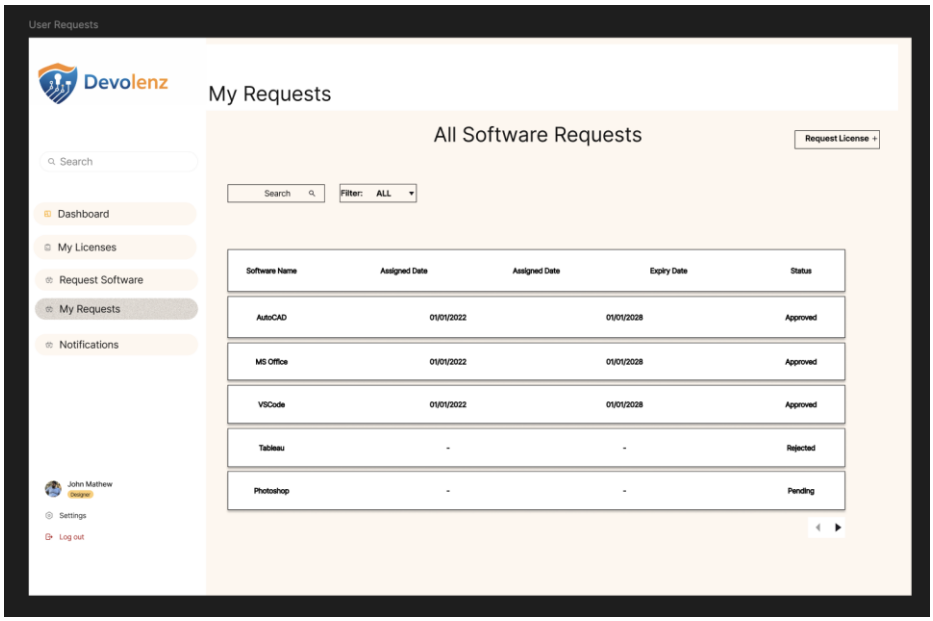
This document is copyright protected. Do not copy. Do not share (online and offline).

9. My Requests Page

This page allows users to track all their submitted requests:

- List of requested software
- Request and approval dates
- Status (Pending, Approved, Rejected)

Status tracking improves transparency and keeps users informed about their requests.



10. Navigation & Layout Structure

Across all screens:

- A **left sidebar navigation** ensures consistent access to features
- Top sections highlight page titles and actions
- Soft color palette (beige/neutral tones) improves readability

Consistency in navigation reduces learning time and enhances usability.

This document is copyright protected. Do not copy. Do not share (online and offline).

4. Conclusion

The Software License Management System addresses key challenges in managing software assets, including lack of visibility, inefficient manual tracking, and improper license allocation. The system introduces a centralized platform with role-based access, enabling admins to manage licenses and users while allowing users to request and track software efficiently.

The use of SysML diagrams ensured clear requirement modelling, while JIRA supported structured project planning and task management. The Figma-based UI design enhanced usability through a clean layout, consistent navigation, and intuitive workflows.

Despite its strengths, the system has limitations. It does not currently support automation features such as auto-renewal of licenses, integration with external enterprise systems, or advanced analytics. Additionally, scalability and security aspects can be further improved for real-world deployment.

In conclusion, the project successfully demonstrates an effective solution for managing software licenses in an organized and user-friendly manner. It improves operational efficiency by automating license requests, assignments, and tracking, while reducing errors associated with manual processes.

The project also highlights the importance of combining system design (SysML), project management tools (JIRA), and user-centered design (Figma) in developing practical software solutions.

With further enhancements such as system integration, automation, and improved scalability, the Software License Management System can be extended into a robust enterprise-level application.

This document is copyright protected. Do not copy. Do not share (online and offline).

Use of Gen-AI

Generative AI (**ChatGPT (OpenAI)**) was used to assist in documentation, UI/UX design suggestions, and explanation of system diagrams. It helped improve clarity, structure, and efficiency in project development. All outputs were reviewed and refined to ensure accuracy and relevance.

Prompts were used to refine report content, design system diagrams, structure JIRA tasks, and generate UI/UX ideas. The prompts were iterative, and all outputs were reviewed and adapted to match project requirements.

Examples:

1. Clarification of concepts
   "Difference between include and extend in use case diagram"
2. Guidance on system design and diagrams
   e.g., "Explain SysML requirement diagram structure for my project"

AI-generated content was reviewed against lecture materials and assignment requirements, then adapted to match the project workflow and scope. Irrelevant features were removed, and content was refined for accuracy and consistency.

Please note that reporting the use of Gen-AI is mandatory. Failure to disclose the use of Gen-AI tools may be considered a breach of academic integrity policies.

Reflection

Through this assignment, I gained a deeper understanding of the **software development lifecycle**, particularly the importance of requirement analysis, system design, and structured project management. I learned how to translate real-world problems into system requirements and represent them using **SysML diagrams**, which helped in visualising system structure, behaviour, and relationships clearly.

Working with **JIRA** improved my ability to organise tasks using epics, user stories, and subtasks, and helped me understand how projects are managed in an Agile environment. Additionally, designing the user interface using **Figma** enhanced my understanding of UI/UX principles such as clarity, consistency, and user-friendly navigation.

One of the main challenges I faced was understanding and correctly applying relationships in system diagrams, especially concepts such as «include», «extend», and dependency relationships. Another difficulty was ensuring consistency between different components of the assignment, such as aligning JIRA tasks with system

Queensland University of Technology

This document is copyright protected. Do not copy. Do not share (online and offline).

requirements and UI design. Structuring the workflow logically, particularly for request handling and license assignment, also required careful planning.

These challenges were addressed through practice, referring to tutorial materials, and refining the design iteratively. Overall, this assignment helped me develop both technical and analytical skills, provided practical insight into designing and managing a real-world software system.

References:

ServiceNow Software Asset Management. *Enterprise-Level Tools* https://www.servicenow.com

Figma Community Dashboard Template. *Full Dashboard with Widgets, Sidebar, Charts & Graphs*. https://www.figma.com/community/file/1292982580121452290

This document is copyright protected. Do not copy. Do not share (online and offline).